

Practical No. 1

Definition : Write a program to demonstrate decision-making statements using if, if-else, and if-elif-else in Python.

Aim/ Objective:

- To understand the use of conditional statements in Python.
- To learn how to use if, if-else, and if-elif-else statements.
- To perform decision-making based on conditions.
- To understand how conditions control program execution.

Source Code:

```
age = 20
if age >= 18:
    print("You are eligible to vote.")

number = 7
if number % 2 == 0:
    print("The number is even.")
else:
    print("The number is odd.")

marks = 85
if marks >= 90:
    print("Grade: A+")
elif marks >= 75:
    print("Grade: A")
elif marks >= 60:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Grade: F")
```

Output:

```
You are eligible to vote.
The number is odd.
Grade: A
```

Conclusion:

The program successfully demonstrates decision-making statements in Python. It shows how if, if-else, and if-elif-else statements are used to evaluate conditions and execute appropriate blocks of code.

Practical No. 2

Definition : Write a program to check whether a number is positive, negative, or zero using conditional statements.

Aim/ Objective:

- To understand nested conditional statements in Python.
- To check whether a number is positive, negative, or zero.
- To learn how multiple conditions can be evaluated.
- To develop basic decision-making skills using Python.

Source Code:

```
number = int(input("Enter a number: "))

if number >= 0:
    if number == 0:
        print("The number is zero.")
    else:
        print("The number is positive.")
else:
    print("The number is negative.")
```

Output:

```
Enter a number: 25
The number is positive.
```

Conclusion:

The program successfully demonstrates the use of nested conditional statements. It checks a number and correctly identifies whether it is positive, negative, or zero.

Practical No. 3

Definition : Write a program to generate a multiplication table of a given number using a for loop.

Aim/ Objective:

- To understand the use of for loops in Python.
- To learn how to repeat a block of statements.
- To generate a multiplication table using iteration.
- To understand the use of range() with a loop.

Source Code:

```
number = int(input("Enter a number: "))  
  
for i in range(1, 11):  
    print(number, "x", i, "=", number * i)
```

Output:

```
Enter a number: 5  
5 x 1 = 5  
5 x 2 = 10  
5 x 3 = 15  
5 x 4 = 20  
5 x 5 = 25  
5 x 6 = 30  
5 x 7 = 35  
5 x 8 = 40  
5 x 9 = 45  
5 x 10 = 50
```

Conclusion:

The program successfully demonstrates iteration using a for loop. It uses the range() function to generate values from 1 to 10 and prints the multiplication table of the given number.

Practical No. 4

Definition : Write a program to calculate the sum of digits of a number using a while loop.

Aim/ Objective:

- To understand the use of while loops in Python.
- To learn how to process digits of a number repeatedly.
- To calculate the sum of digits using arithmetic operations.
- To understand loop control using a changing condition.

Source Code:

```
number = int(input("Enter a number: "))

sum = 0
while number > 0:
    digit = number % 10
    sum = sum + digit
    number = number // 10

print("Sum of digits =", sum)
```

Output:

```
Enter a number: 12345
Sum of digits = 15
```

Conclusion:

The program successfully demonstrates the use of a while loop to process each digit of a number. It extracts digits using modulus and reduces the number using integer division to calculate the total sum.

Practical No. 5

Definition : Write a program to demonstrate the use of break, continue, and pass statements in Python.

Aim/ Objective:

- To understand the purpose of break, continue, and pass.
- To learn how break terminates a loop.
- To learn how continue skips the current iteration.
- To understand how pass is used as a placeholder statement.

Source Code:

```
print("Break statement:")
for i in range(1, 6):
    if i == 4:
        break
    print(i)

print("\nContinue statement:")
for i in range(1, 6):
    if i == 3:
        continue
    print(i)

print("\nPass statement:")
for i in range(1, 6):
    if i == 3:
        pass
    print(i)
```

Output:

```
Break statement:
1
2
3

Continue statement:
1
2
4
5

Pass statement:
1
2
3
4
5
```

Conclusion:

The program successfully demonstrates the break, continue, and pass statements. It shows how break stops a loop, continue skips an iteration, and pass allows a statement block to remain syntactically valid without performing an operation.

Practical No. 6

Definition : Write a program to demonstrate iteration through lists, strings, and dictionaries using for loops.

Aim/ Objective:

- To understand iteration over different Python data structures.
- To iterate through list elements using a for loop.
- To iterate through characters of a string.
- To iterate through dictionary keys and values.

Source Code:

```
fruits = ["Apple", "Banana", "Mango", "Orange"]

print("List elements:")
for fruit in fruits:
    print(fruit)

name = "Python"
print("\nString characters:")
for character in name:
    print(character)

student = {
    "Name": "Rahul",
    "Age": 20,
    "Course": "Python"
}

print("\nDictionary elements:")
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
List elements:
Apple
Banana
Mango
Orange

String characters:
P
y
t
h
o
n

Dictionary elements:
Name : Rahul
Age : 20
Course : Python
```

Conclusion:

The program successfully demonstrates iteration over lists, strings, and dictionaries. It shows how for loops can be used to access elements and key-value pairs from different Python data structures.

Practical No. 7

Definition : Write a program to demonstrate list, dictionary, and set comprehensions in Python.

Aim/ Objective:

- To understand the concept of comprehensions in Python.
- To create lists efficiently using list comprehension.
- To create dictionaries using dictionary comprehension.
- To create sets using set comprehension.

Source Code:

```
squares = [x * x for x in range(1, 6)]
print("List comprehension:", squares)

squares_dict = {x: x * x for x in range(1, 6)}
print("Dictionary comprehension:", squares_dict)

squares_set = {x * x for x in range(1, 6)}
print("Set comprehension:", squares_set)
```

Output:

```
List comprehension: [1, 4, 9, 16, 25]
Dictionary comprehension: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
Set comprehension: {1, 4, 9, 16, 25}
```

Conclusion:

The program successfully demonstrates list, dictionary, and set comprehensions. It shows how comprehensions provide a concise way to create collections from an iterable.

Practical No. 8

Definition : Write a program to demonstrate global, local, and nonlocal variables in Python.

Aim/ Objective:

- To understand different variable scopes in Python.
- To understand the use of global variables.
- To demonstrate local variables inside functions.
- To demonstrate nonlocal variables in nested functions.

Source Code:

```
# Global
x = 10

def outer():
    # Nonlocal
    y = 20

    def inner():
        # Accessing nonlocal variable
        nonlocal y
        y = y + 5
        print("Nonlocal variable:", y)

    inner()

def demo():
    # Local
    z = 30
    print("Local variable:", z)

    # Accessing global variable
    print("Global variable:", x)

# Calling functions
demo()
outer()
```

Output:

```
Local variable: 30
Global variable: 10
Nonlocal variable: 25
```

Conclusion:

The program successfully demonstrates Python variable scope. It shows the use of global, local, and nonlocal variables and how variables behave inside functions and nested functions.

Practical No. 9

Definition : Write a program to demonstrate iterable and iterator objects using `iter()` and `next()`.

Aim/ Objective:

- To understand the concept of iterable objects.
- To understand the concept of iterators in Python.
- To create an iterator using the `iter()` function.
- To retrieve elements from an iterator using `next()`.

Source Code:

```
# List is an iterable
numbers = [10, 20, 30, 40, 50]

print("Iterable:")
for number in numbers:
    print(number)

# Creating an iterator from the list
iterator = iter(numbers)

print("\nIterator:")
print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

Output:

```
Iterable:
10
20
30
40
50

Iterator:
10
20
30
40
50
```

Conclusion:

The program successfully demonstrates the difference between an iterable and an iterator. It shows how a list can be iterated using a for loop and how `iter()` and `next()` can be used to access iterator elements sequentially.

Practical No. 10

Definition : Write a program to demonstrate generators using the yield statement.

Aim/ Objective:

- To understand the concept of generators in Python.
- To learn how the yield statement works.
- To generate values one at a time.
- To understand how a generator can be iterated using a for loop.

Source Code:

```
def generate_numbers(n):  
    for i in range(1, n + 1):  
        yield i  
  
numbers = generate_numbers(5)  
  
print("Sequence of numbers:")  
for number in numbers:  
    print(number)
```

Output:

```
Sequence of numbers:  
1  
2  
3  
4  
5
```

Conclusion:

The program successfully demonstrates the use of generators in Python. The yield statement produces values one at a time, and the generator can be traversed using a for loop.